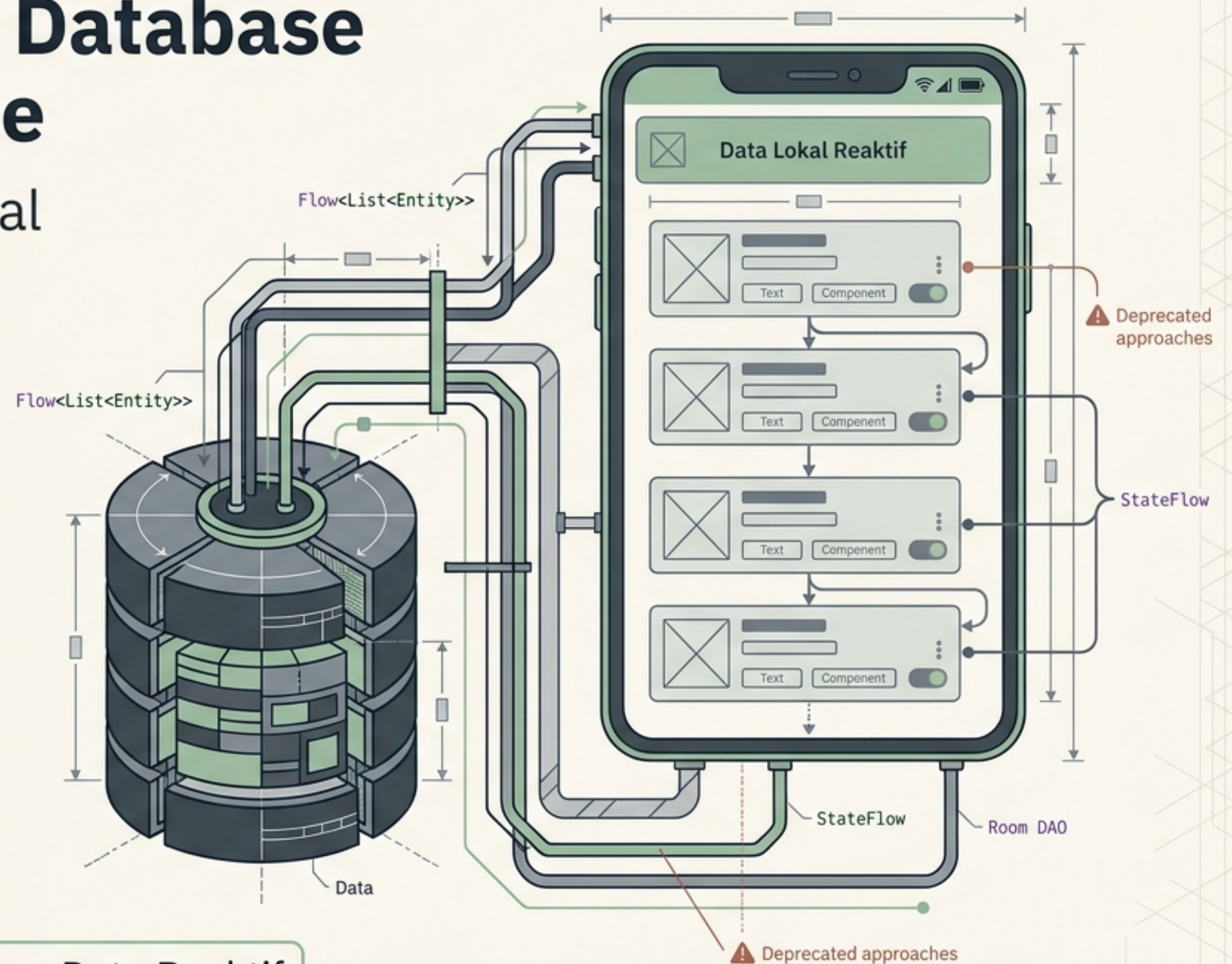


Penguasaan Room Database & Jetpack Compose

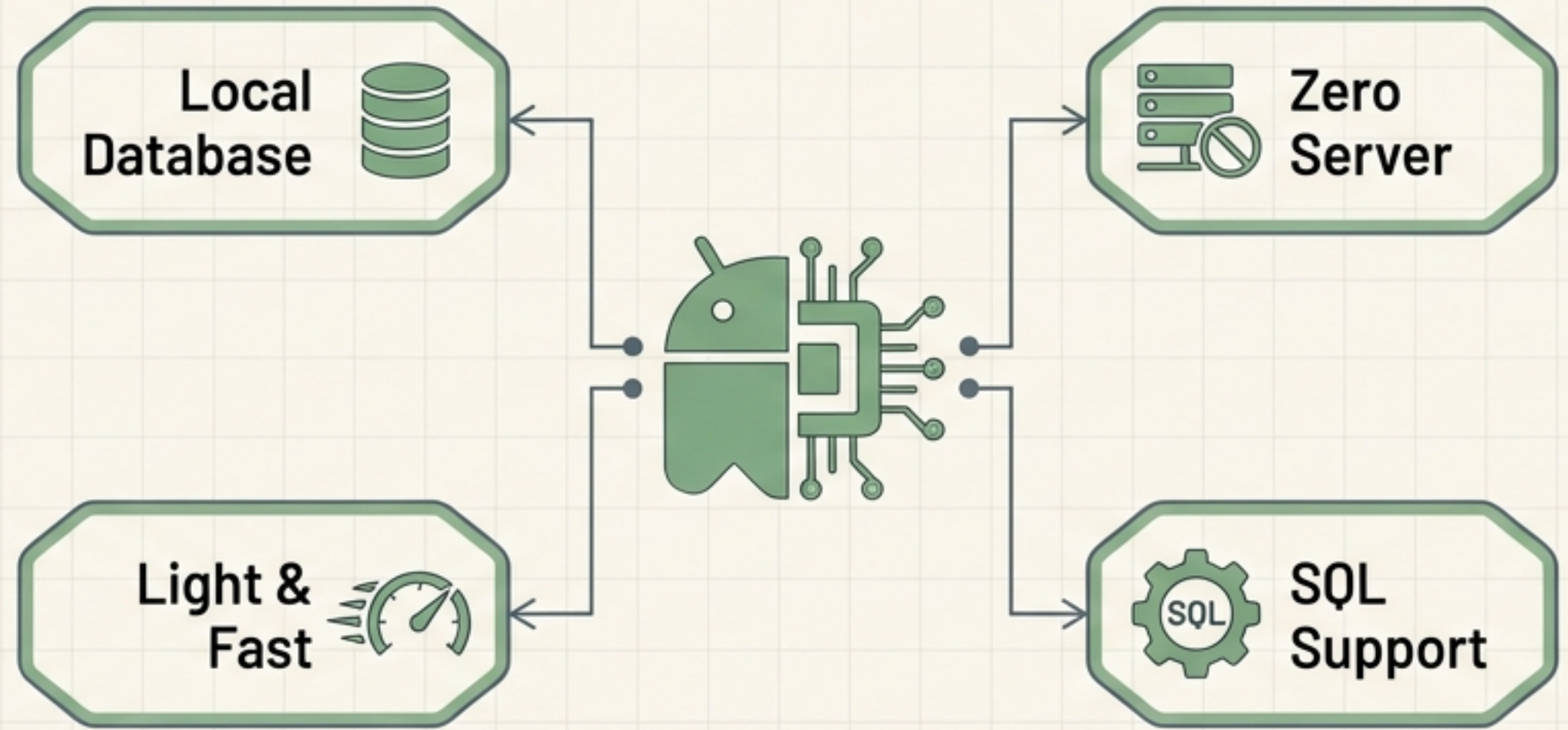
Arsitektur Penyimpanan Lokal Modern untuk Android



Dari Kompleksitas SQLite menuju Aliran Data Reaktif

Fondasi Penyimpanan Lokal Android

SQLite adalah database relasional ringan yang tertanam langsung di sistem Android. Tidak memerlukan server dan bekerja sangat cepat untuk menyimpan data pengguna secara permanen.



● `sqlite_data.db`

id	nama	email
1	Andi	andi@gmail.com
2	Budi	budi@gmail.com

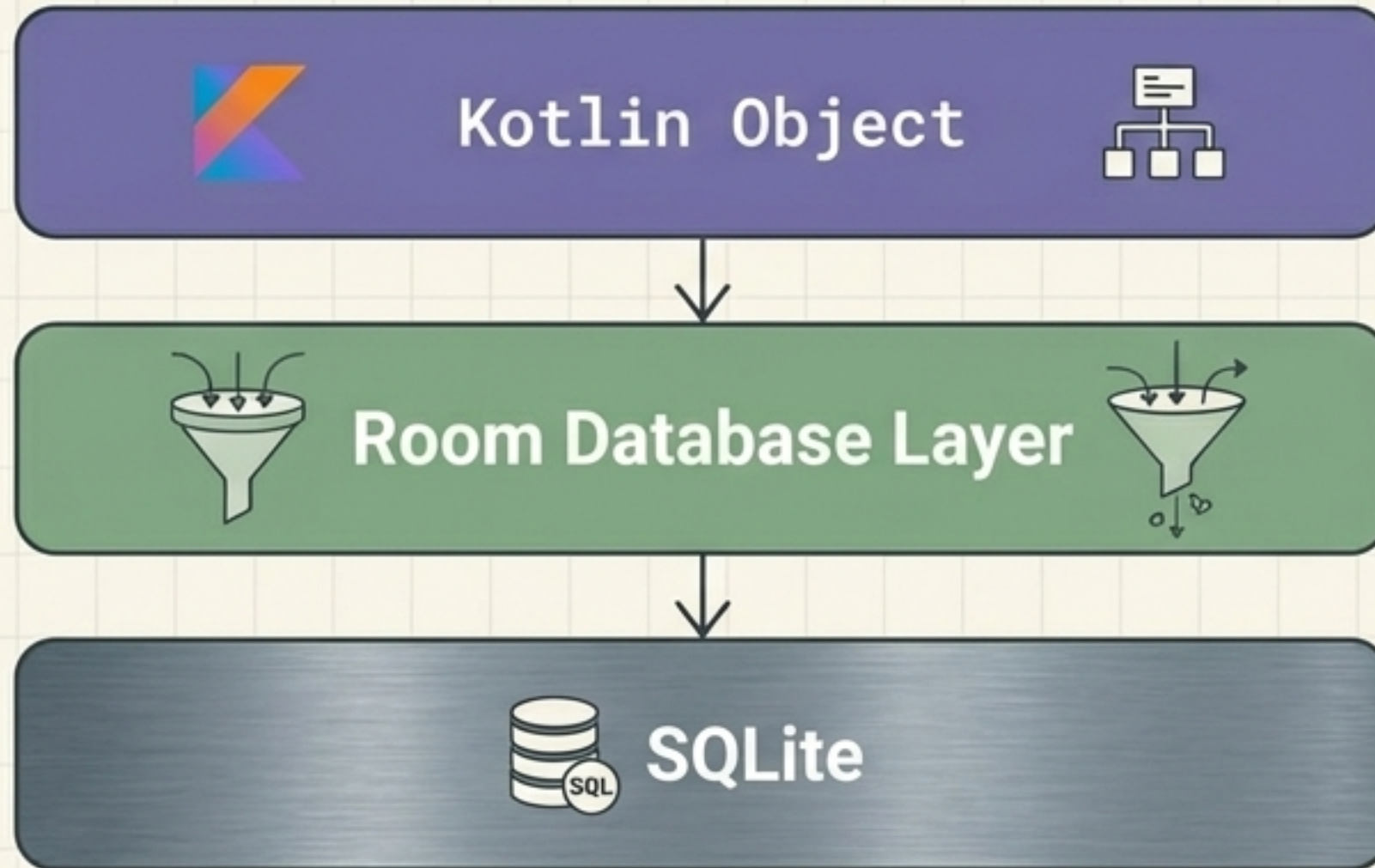
Mengapa Meninggalkan SQLite Manual?

	SQLite Manual (Masa Lalu)	Kebutuhan Modern (Room)
Query SQL	Ditulis manual sebagai String murni (Rawan typo)	Membutuhkan validasi saat compile (Type-safe)
Kode	Boilerplate panjang & berulang	Kode bersih & terstruktur
Eksekusi	Sinkron, rawan membuat UI freeze	Asynchronous (Aman untuk Main Thread)
Observasi Data	Pasif (Harus ditarik ulang)	Reaktif (UI update otomatis)

```
INSERT INTO siswa(nama,email)VALUES('Andi',...
```


Abstraksi Cerdas dengan Room Database

Room hadir sebagai library persistence dari Android Jetpack. Room bukan pengganti SQLite, melainkan abstraction layer yang mengelola SQLite dengan pendekatan Object-Oriented.



Type-Safe

Mencegah error sebelum compile



Reactive

Mendukung LiveData & Flow

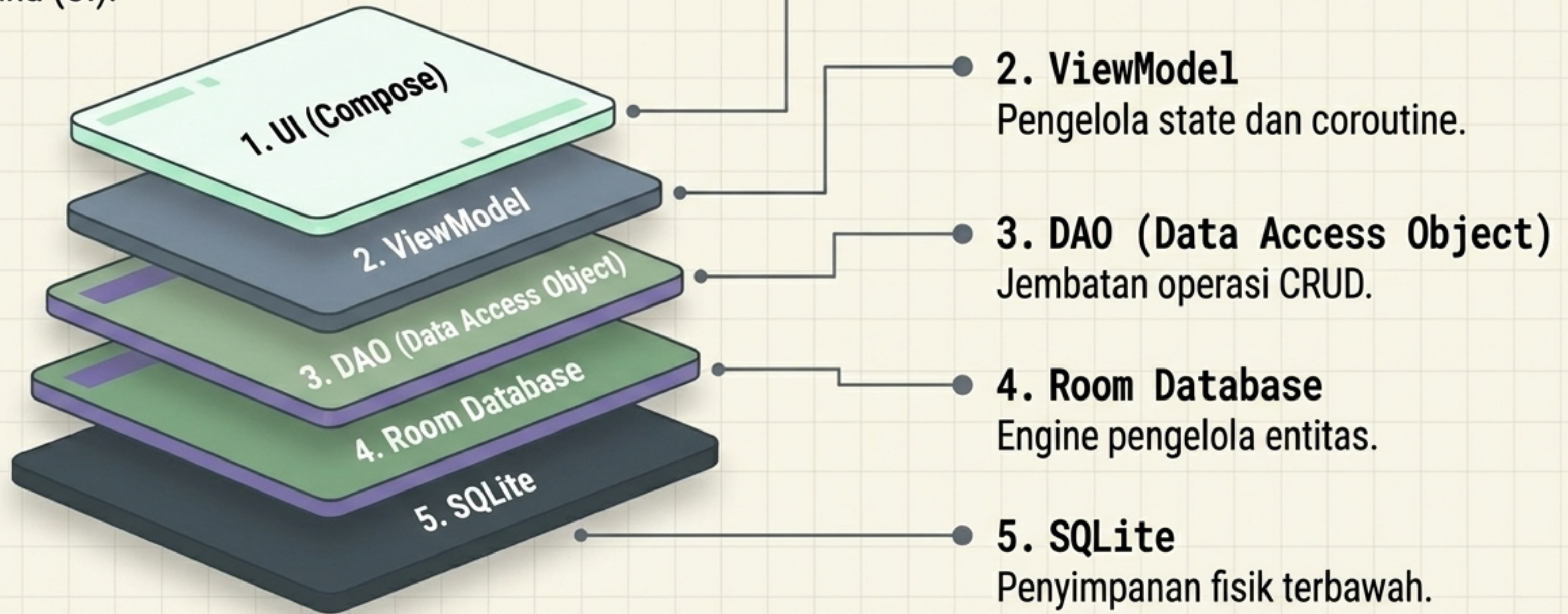


Compose Ready

Integrasi mulus dengan UI modern

Arsitektur Lapis Pengelolaan Data

Setiap lapisan memiliki tanggung jawab spesifik. Room mengisolasi kompleksitas penyimpanan dari antarmuka pengguna (UI).



Konfigurasi Proyek & Dependensi

```
// build.gradle.kts (Module: app)
plugins {
    id("kotlin-kapt")
}

dependencies {
    val room_version = "2.6.1"
    implementation("androidx.room:room-runtime:$room_version")
    kapt("androidx.room:room-compiler:$room_version")
    implementation("androidx.room:room-ktx:$room_version")
}
```



Diperlukan untuk men-generate kode boilerplate Room secara otomatis saat compile.

Pilar 1 - Anatomi Entity

Entity adalah representasi kelas data yang akan diterjemahkan oleh Room menjadi struktur tabel fisik di dalam SQLite.

```
// data/entity/Siswa.kt
import androidx.room.Entity
import androidx.room.PrimaryKey
```

```
@Entity(tableName = "siswa")
data class Siswa(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val nama: String,
    val email: String
)
```

Tabel: siswa		
id	nama	email

Pilar 2 - Kontrak Operasi Data (DAO)

DAO (Data Access Object) adalah antarmuka tempat kita mendefinisikan seluruh operasi interaksi dengan database tanpa perlu menulis logika implementasinya.

Menyimpan objek menjadi baris baru.

```
@Dao
interface SiswaDao {
    @Insert
    suspend fun insertSiswa(siswa: Siswa)

    @Update
    suspend fun updateSiswa(siswa: Siswa)

    @Delete
    suspend fun deleteSiswa(siswa: Siswa)

    @Query("SELECT * FROM siswa")
    fun getAllSiswa(): Flow<List<Siswa>>
}
```

Memodifikasi baris berdasarkan Primary Key.

Menghapus data siswa.

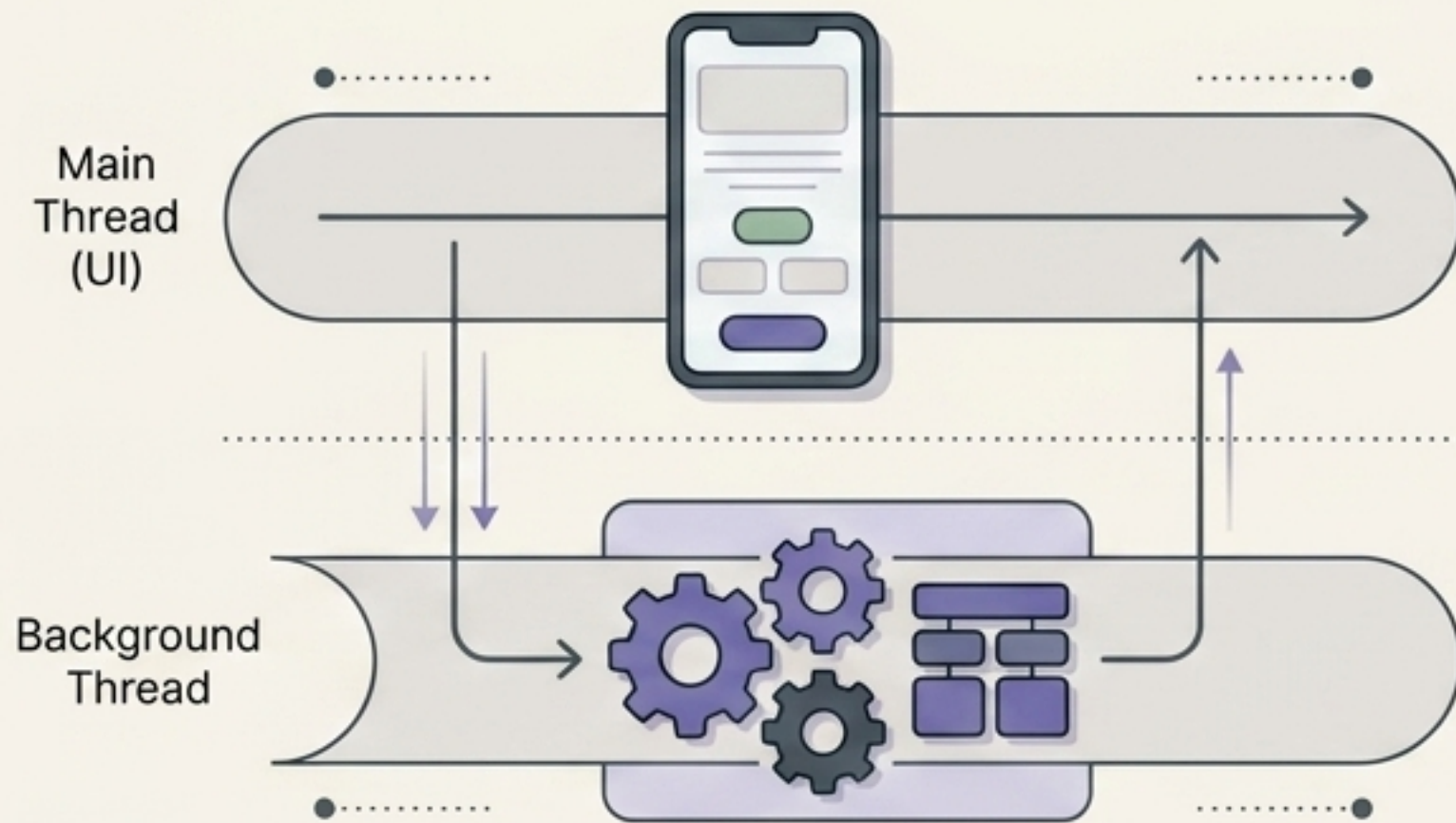
Mendukung SQL kustom untuk pembacaan data kompleks.

Detak Jantung Aplikasi: Asynchronous & Reactive

suspend

Operasi Tulis (Insert/Update/Delete)

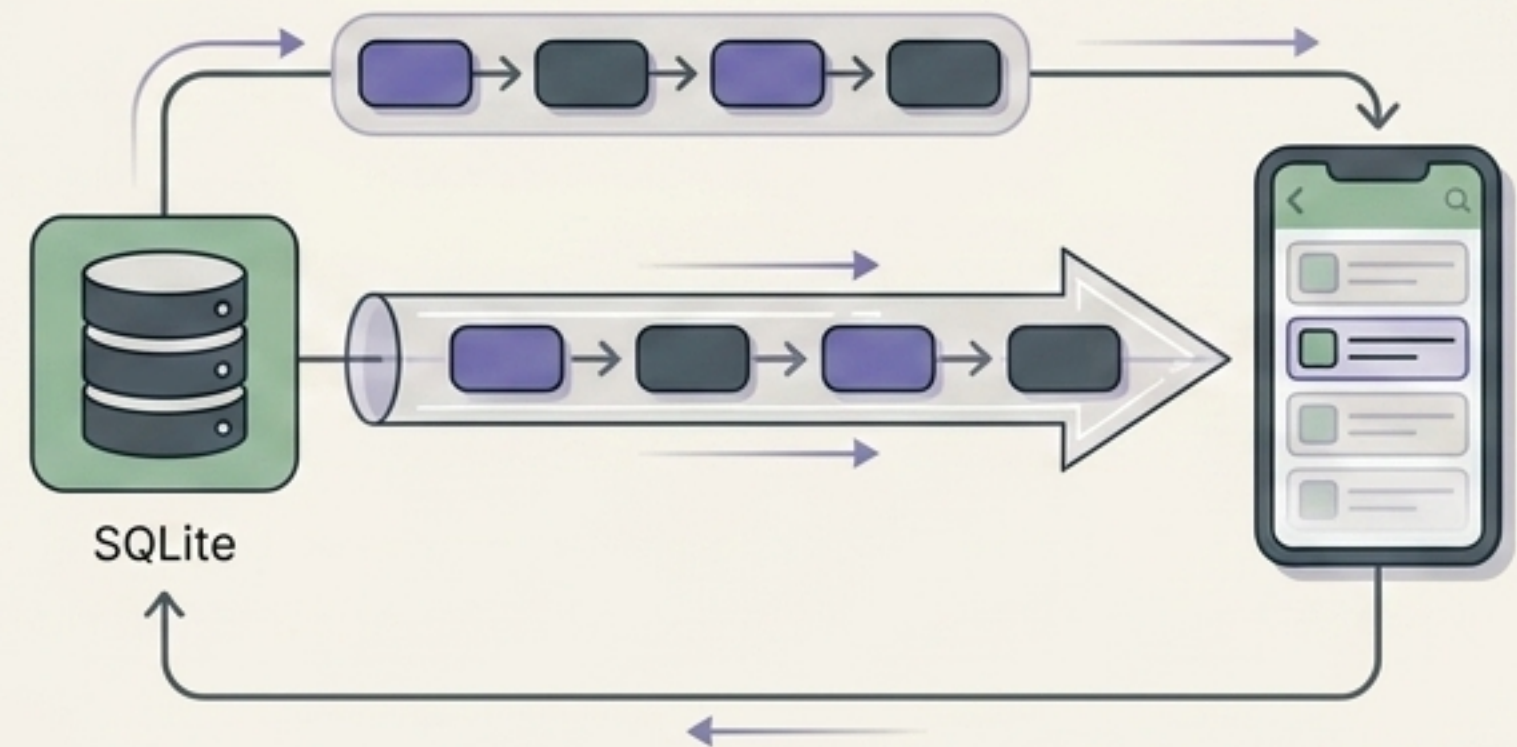
Berjalan secara asynchronous di background thread untuk mencegah UI Freeze (Aplikasi not responding).



Flow<T>

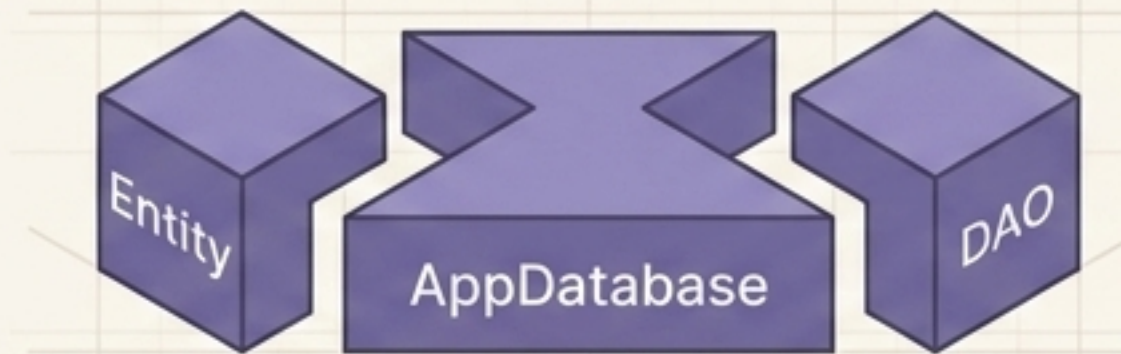
Operasi Baca (Query)

Menciptakan aliran data **reaktif**. Jika data di SQLite berubah, antarmuka Compose akan melakukan update secara instan dan otomatis (real-time).



Pilar 3 - Menyatukan Komponen (Database Class)

Kelas utama ini menjadi **pusat koneksi**. Menggunakan pola Singleton memastikan hanya ada satu instance database yang aktif di memori untuk mencegah konflik antrean data.



```
@Database(entities = [Siswa::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun siswaDao(): SiswaDao
    companion object {
        @Volatile
        private var INSTANCE: AppDatabase? = null
    }
    ...
}
```

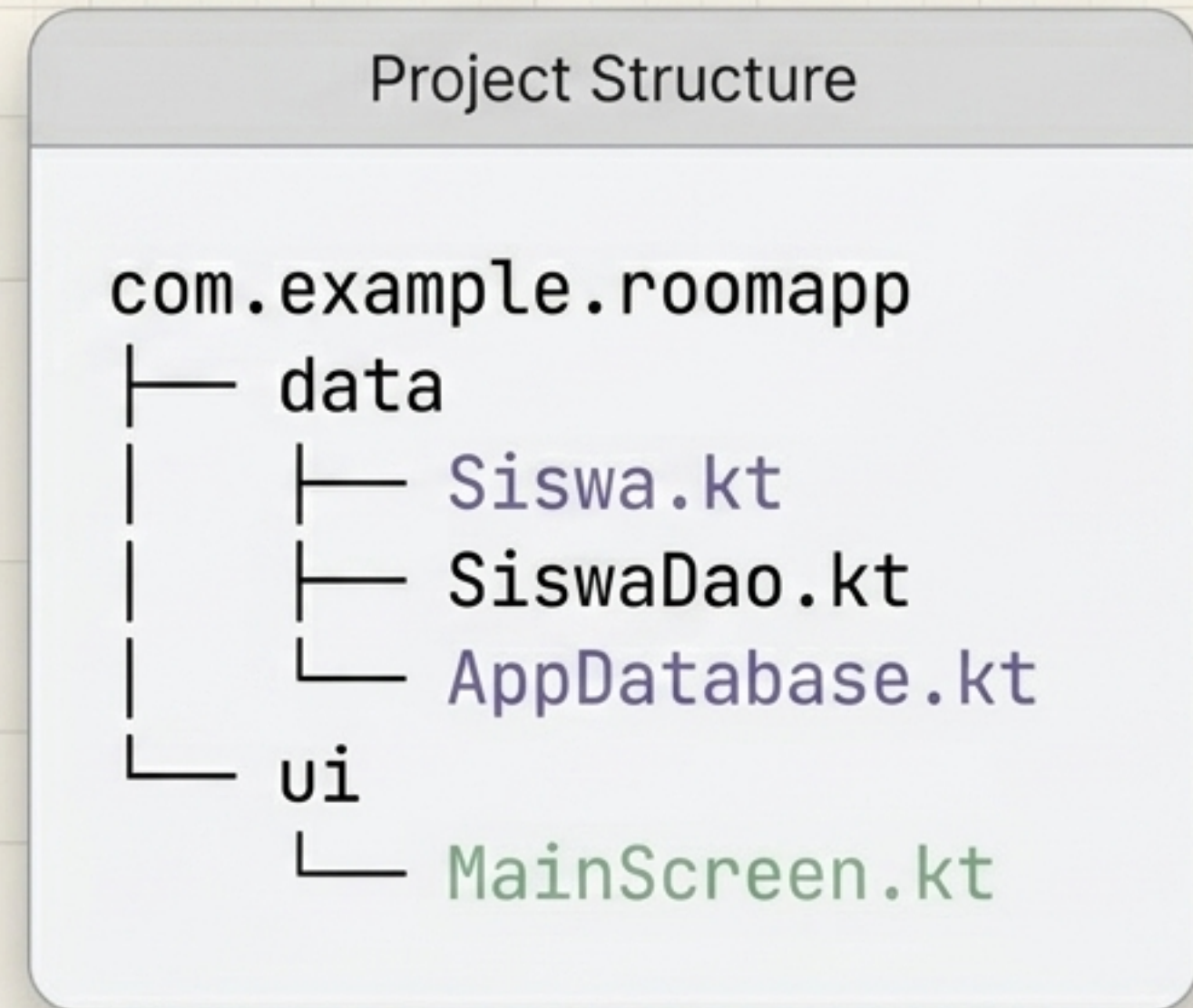
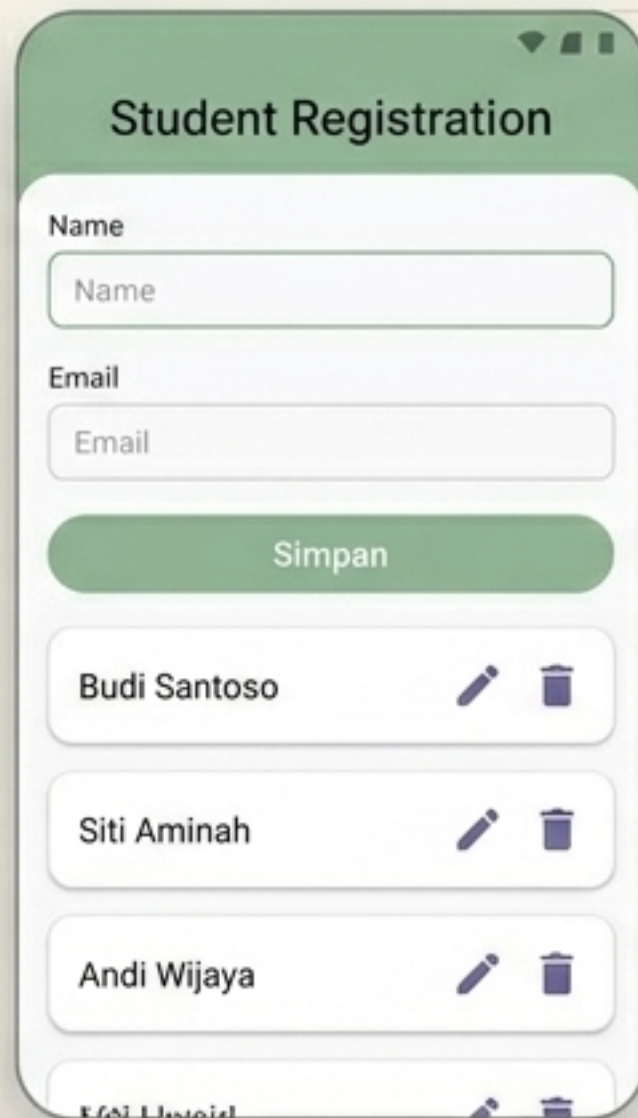
1 Mendaftarkan semua tabel.

2 Kelas induk bawaan Room.

3 Pola Singleton: Mencegah duplikasi di memori.

Studi Kasus: Aplikasi Registrasi Siswa

Menerapkan arsitektur **Room** ke dalam skenario nyata: Aplikasi pencatatan data siswa dengan kapabilitas CRUD lengkap.



Integrasi Data dengan Antarmuka Compose

Membaca Data (Reactive)

```
val siswaList by dao.getAllSiswa().collectAsState(initial = emptyList())
```

collectAsState mengubah aliran Flow dari Room menjadi State yang dipantau lipantau otomatis oleh antarmuka Compose.

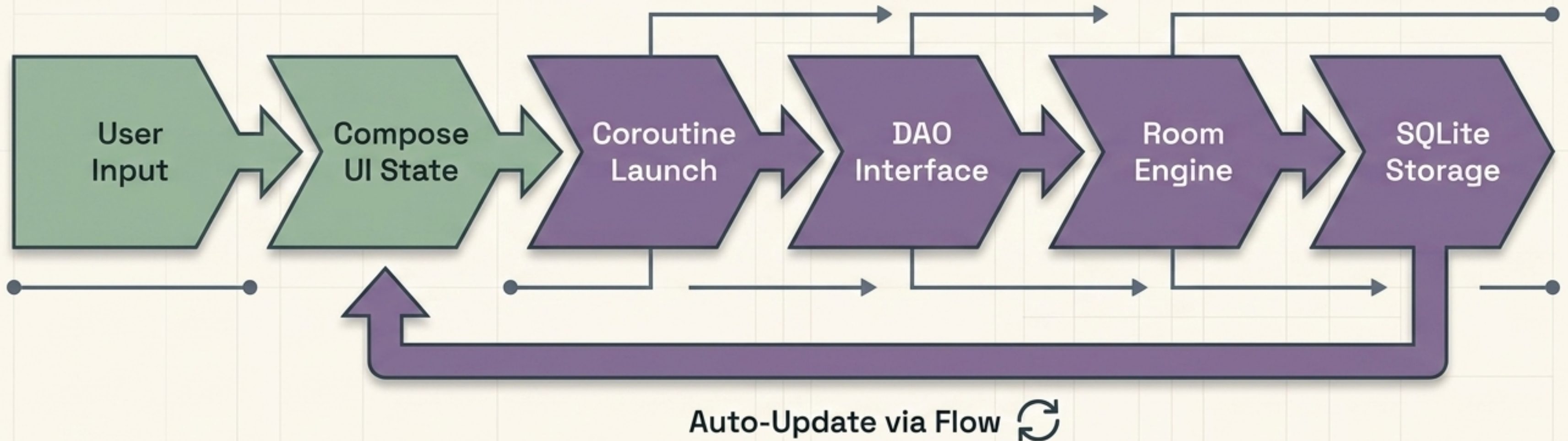
Menulis Data (Async)

```
coroutineScope.launch {  
    dao.insertSiswa(Siswa(nama = inputNama, email = inputEmail))  
}
```

coroutineScope.launch digunakan untuk memanggil fungsi suspend dengan aman dari dalam event onClick tombol, tanpa mengganggu Main Thread.

Sintesis: Alur Transformasi Data End-to-End

Proses ini berputar dalam siklus. Saat data berhasil disimpan ke SQLite, Room mendeteksi perubahan, mengalirkan sinyal via Flow kembali ke Compose UI, dan daftar layar ter-update seketika tanpa perlu refresh manual.



Sinergi Room & Jetpack Compose

Fitur Arsitektur	Dampak Langsung pada UI
✓ Reactive Data (Flow)	Update antarmuka 100% otomatis dengan collectAsState.
✓ Coroutine Support	Operasi background mulus tanpa aplikasi nge-lag.
✓ Type Safety	Bebas dari crash mendadak akibat typo pada query SQL.
✓ SQLite Abstraction	Pengembangan fitur baru jauh lebih cepat.
✓ Cleaner Code	Struktur proyek mudah dipahami dan maintainable oleh tim.

Tantangan Penguasaan Materi

